

Answer the questions

Question 1: What is the default value assigned to array elements in C#?

- In C#, array elements are automatically initialized to the **default value of their data type**.

Examples:

- Int → 0
- Double → 0.0
- bool → false
- char → '\0'
- Reference type like string → null

- Answer:** The default value depends on the element's data type.

Question 2: What is the difference between Array.Clone() and Array.Copy()?

- The main difference is **what they return and how much control you have over the copying**:

Array.Clone()	Array.Copy()
Creates a copy of the entire array	Copies elements from one array to another
Returns a new array as <code>object</code>	Returns <code>void</code>
No need to specify source/destination positions	You can specify source, destination, and number of elements
Performs a shallow copy	Also copies values/references; for reference types, it is effectively a shallow copy

Example:

Clone():

```
</> C#  
  
int[] arr1 = { 1, 2, 3 };  
int[] arr2 = (int[])arr1.Clone();
```

Copy():

```
</> C#  
  
int[] arr1 = { 1, 2, 3 };  
int[] arr2 = new int[3];  
  
Array.Copy(arr1, arr2, 3);
```

Clone() → quickly creates a copy of the **whole array**.

Copy() → gives you **more control** over which elements are copied and where.

Question 3: What is the difference between GetLength() and Length for multidimensional arrays?

- For **multidimensional arrays** in C#, the difference is:
 - **Length** → returns the **total number of elements** in all dimensions.
 - **GetLength(dimension)** → returns the number of elements in a **specific dimension**.

Example:

```
</> C#
```

```
int[,] arr = new int[3, 4];
```



This array has 3 rows × 4 columns = 12 elements.

```
</> C#
```

```
Console.WriteLine(arr.Length);           // 12
Console.WriteLine(arr.GetLength(0));     // 3
Console.WriteLine(arr.GetLength(1));     // 4
```



Question 4: What is the difference between Array.Copy() and Array.ConstrainedCopy()?

- The main difference is **what happens if the copy operation fails**.

Array.Copy()	Array.ConstrainedCopy()
Copies elements from one array to another	Also copies elements from one array to another
If an error occurs, the destination array may be partially modified	Guarantees the destination array is not partially modified if the operation fails
Less strict	More reliable/atomic behavior
Commonly used for normal copying	Useful when you need the copy to succeed completely or not at all

Example:

```
</> C#
```

```
int[] source = { 1, 2, 3 };
int[] destination = new int[3];

Array.Copy(source, destination, 3);
```



Using `ConstrainedCopy()`:

```
</> C#
```

```
Array.ConstrainedCopy(source, 0, destination, 0, 3);
```



Array.Copy() → Normal copy, destination **may be partially changed** if an exception occurs.

Array.ConstrainedCopy() → **All-or-nothing** copy; if it can't complete, the destination is left unchanged.

Question 5: Why is foreach preferred for read-only operations on arrays?

- `foreach` is often preferred for **read-only operations** because it is **simple, safe, and avoids index management**.

Example:

```
</> C#  
  
int[] numbers = { 10, 20, 30, 40 };  
  
foreach (int number in numbers)  
{  
    Console.WriteLine(number);  
}
```

Why use `foreach`?

- No index needed → You don't have to use `numbers[i]`.
- Less chance of errors → Avoids mistakes like accessing an invalid index.
- **Readable** → Clearly shows that you're simply visiting each element.
- Good for read-only operations → You can access each element without modifying the array through the loop variable.

Question 6: Why is input validation important when working with user inputs?

- **Input validation** is important because it ensures that the data entered by the user is **valid, expected, and safe** before the program processes it.

Why is it important?

- Prevents errors and crashes — e.g., entering "abc" when an integer is expected.
- **Ensures correct results** — prevents invalid values from affecting calculations.
- **Improves user experience** — tells the user what they need to enter.
- **Improves security** — helps prevent malicious or unexpected input.

Question 7: How can you format the output of a 2D array for better readability?

- You can format a 2D array using nested loops and add spaces or tabs between elements so that rows and columns are aligned.
- answer: Use nested for loops to iterate through rows and columns, and use spacing, tabs (`\t`), or formatted string alignment to make the output easier to read.

Question 8: When should you prefer a switch statement over if-else?

- You should prefer a **switch statement** when you need to compare **one variable/expression against multiple specific values**.

Example:

```
</> C#  
  
int day = 3;  
  
switch (day)  
{  
    case 1:  
        Console.WriteLine("Monday");  
        break;  
    case 2:  
        Console.WriteLine("Tuesday");  
        break;  
    case 3:  
        Console.WriteLine("Wednesday");  
        break;  
    default:  
        Console.WriteLine("Invalid day");  
        break;  
}
```

Why use switch?

- Cleaner and more readable when there are many possible values.
- Easier to maintain than a long chain of if-else if.
- Works well with **fixed values**, such as integers, characters, strings, and enums.

When to use if-else instead?

- Use if-else when you have **ranges or complex conditions**:

Final answer: Prefer switch when comparing a single expression against multiple specific values. Use if-else when conditions involve ranges, comparisons, or complex logical expressions.

Question 9: What is the time complexity of Array.Sort()?

- **Average case:** $O(n \log n)$
- **Worst case:** $O(n \log n)$
- **Space complexity:** $O(\log n)$ approximately, depending on the implementation.

Final answer: `Array.Sort()` has a time complexity of $O(n \log n)$.

Question 10: Which loop (for or foreach) is more efficient for calculating the sum of an array, and why?

- For calculating the sum of an array, **for and foreach are generally very similar in performance**. The choice usually depends more on readability than efficiency.

Example:

for:

```
</> C#  
  
int sum = 0;  
  
for (int i = 0; i < numbers.Length; i++)  
{  
    sum += numbers[i];  
}
```

foreach:

```
</> C#  
  
int sum = 0;  
  
foreach (int number in numbers)  
{  
    sum += number;  
}
```

Which is more efficient?

- For a normal **one-dimensional array**, `for` is often considered **slightly more efficient** because it directly accesses elements by index and has very little iteration overhead.
- However, the difference is usually **negligible**, and `foreach` is often preferred because the code is simpler and easier to read when you don't need the index.

Final **answer**: `for` can be slightly more efficient for arrays because it uses direct index access, but both `for` and `foreach` have **O(n)** time complexity. `foreach` is often preferred for readability when only the values are needed.